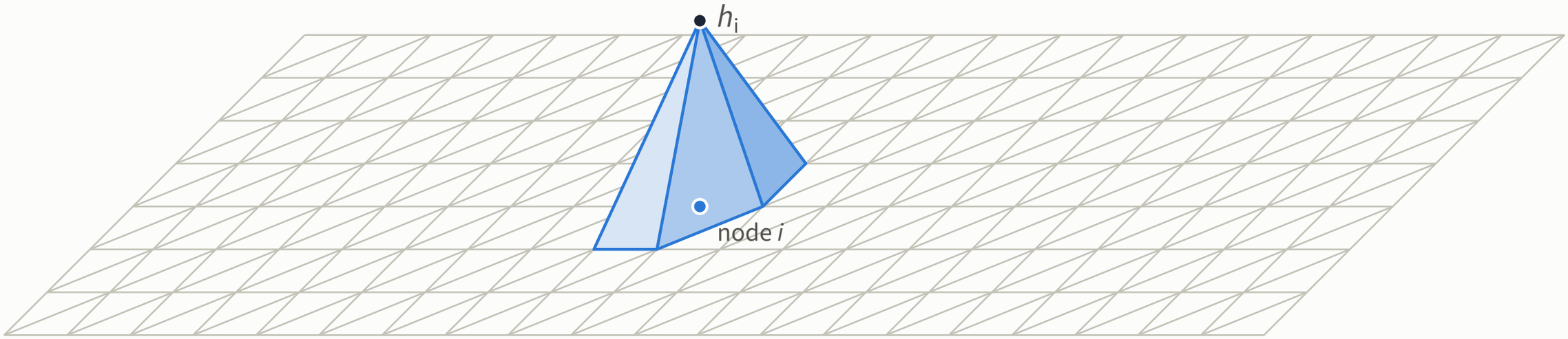
The finite element method

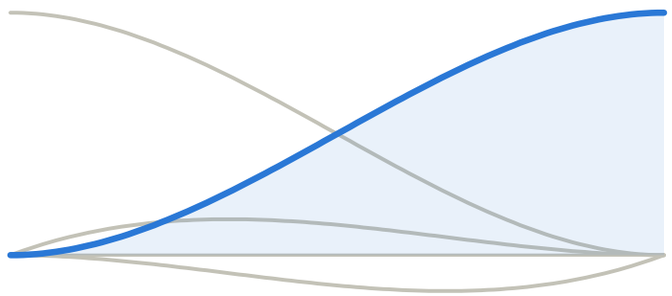
Local basis functions, sparse matrices and the conjugate gradient method



Global or local basis functions

The choice of $h_1, \ldots, h_N$ decides how well u_h approximates u and what the matrix K looks like.

Global basis functions

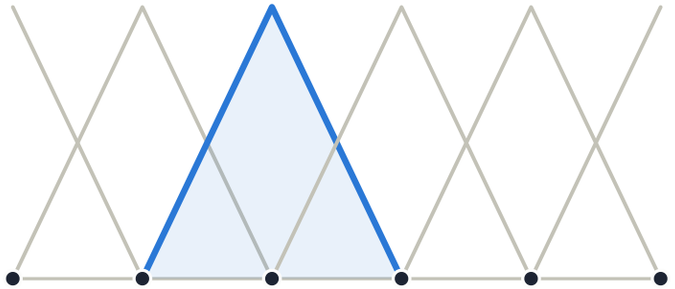




K

nonzero on the whole domain, e.g. the cubic Hermite polynomials of lecture 1. Every $K_{ji} \neq 0$: K is **dense**.

Local basis functions





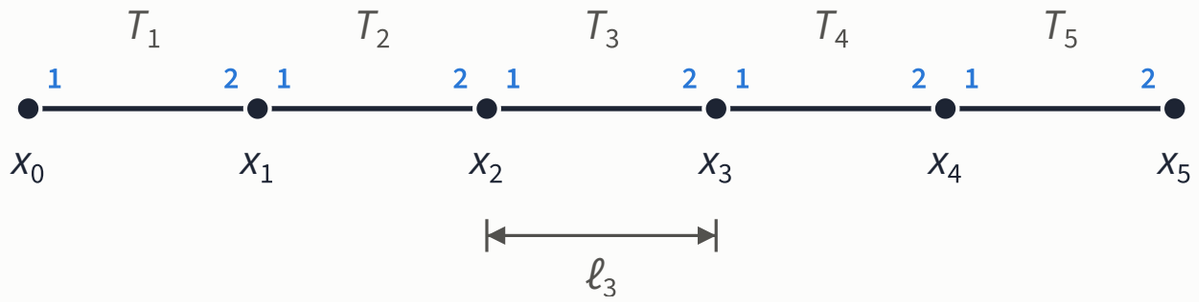
K

nonzero only on a small part of the domain. $a(h_i, h_j) = 0$ if the supports do not overlap: K is **sparse**.

The **finite element method** (FEM) is the Galerkin method with local basis functions.

The mesh and the coincidence table

The domain is approximated by a **mesh**, $\Omega \approx \Omega_h = \bigcup_e T_e$: a union of **elements** T_e (intervals, triangles, tetrahedra) that meet at their vertices, the **nodes**. On each element u_h is a polynomial of low degree.



In 1D the nodes $x_0 < x_1 < \cdots < x_N$ divide the interval into the elements $T_e = [x_{e-1}, x_e]$ of length $\ell_e = x_e - x_{e-1}$. Each element numbers its two nodes locally, 1 (left) and 2 (right).

Coincidence table

the global numbers of the local nodes of every element

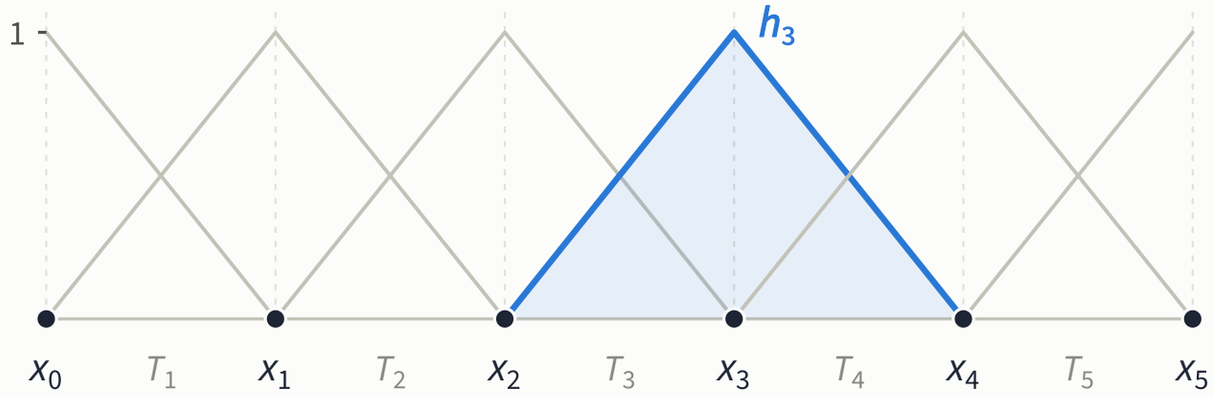
element	local node 1	local node 2
T_1	0	1
T_2	1	2
$\vdots$	$\vdots$	$\vdots$
T_N	$N-1$	N

In FEniCS: `mesh.cells()`

Hat functions in 1D

The hat function of node i:

$$h_i(x) = \begin{cases} (x - x_{i-1})/\ell_i & \text{for } x_{i-1} \le x \le x_i, \\ (x_{i+1} - x)/\ell_{i+1} & \text{for } x_i \le x \le x_{i+1}, \\ 0 & \text{otherwise.} \end{cases}$$



- $h_i(x_j) = \delta_{ij}$, so $U_i = u_h(x_i)$: u_h is the piecewise linear interpolation of the nodal values.
- On T_e only the hat functions of its two nodes are nonzero. In local numbering, h_α is the hat function of the local node α: on T_e, $h_1 = (x_e - x)/\ell_e$ and $h_2 = (x - x_{e-1})/\ell_e$.

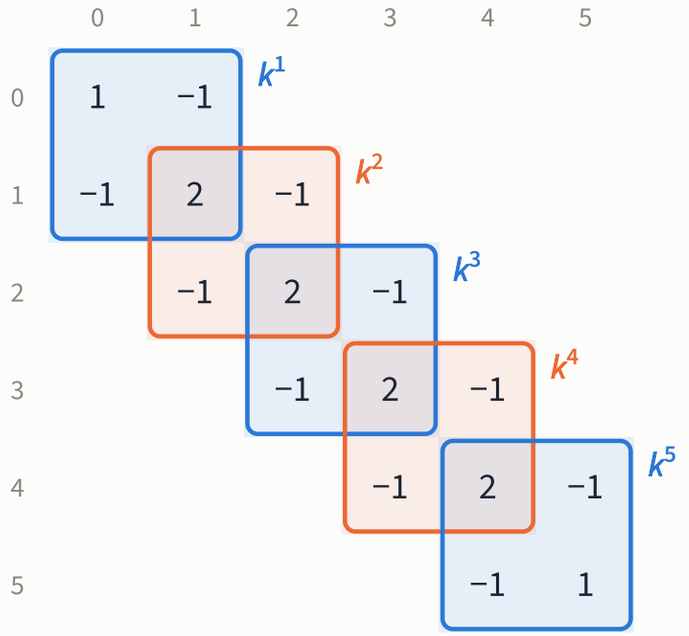
Element matrices and assembly

For $a(u,v) = \int u'\,v'\,dx$ and constant f, on the element T_e:

$$k^e = \left(\int_{T_e} h'_\alpha\, h'_\beta\, dx\right)_{\alpha\beta} = \frac{1}{\ell_e}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

$$f^e = \left(\int_{T_e} f\, h_\alpha\, dx\right)_{\alpha} = \frac{f\,\ell_e}{2}\begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Assembly. K is the sum of the element matrices. The coincidence table says where: $k^e_{\alpha\beta}$ is added to $K_{e-2+\alpha,\ e-2+\beta}$ and f^e_α to $F_{e-2+\alpha}$. Node i shares an element only with $i \pm 1$, so K is **tridiagonal**.



$h\,K$ for 5 elements of length h

The interior rows are the finite difference approximation $(-U_{i-1} + 2U_i - U_{i+1})/h^2$ of $-u''$, multiplied by h. The Dirichlet conditions replace the rows of the boundary nodes.

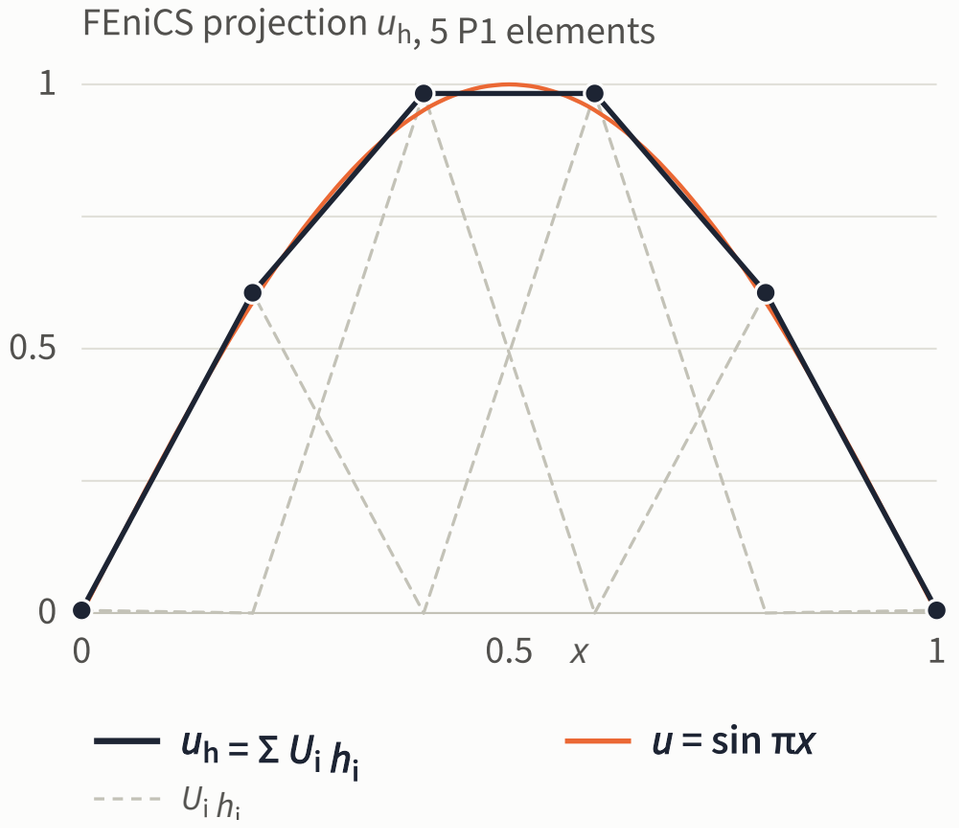
Hat functions and a projection

The Galerkin method with $a(u,v) = \int u\,v\,dx$ and $L(v) = \int \sin(\pi x)\,v\,dx$ gives $M\,U = F$, with the **mass matrix** $M_{ji} = \int h_i\,h_j\,dx$.

```
mesh = IntervalMesh(5, 0.0, 1.0)
V    = FunctionSpace(mesh, 'Lagrange', 1)

u, v = TrialFunction(V), TestFunction(V)
u_ex = Expression('sin(pi*x[0])', degree=4)
M    = assemble(u*v*dx)       # mass matrix
F    = assemble(u_ex*v*dx)    # load vector
uh   = Function(V)
solve(M, uh.vector(), F)

K = assemble(inner(grad(u), grad(v))*dx)
print(K.array())              # tridiagonal, 1/h = 5
```



Unlike the interpolation, u_h does not take the nodal values of u: $U_i = 0.983$ at $x = 0.4$, but $\sin(0.4\pi) = 0.951$. M is assembled from $m^e = \frac{\ell_e}{6}\begin{pmatrix}2 & 1\\ 1 & 2\end{pmatrix}$.

The linear triangle (CST)

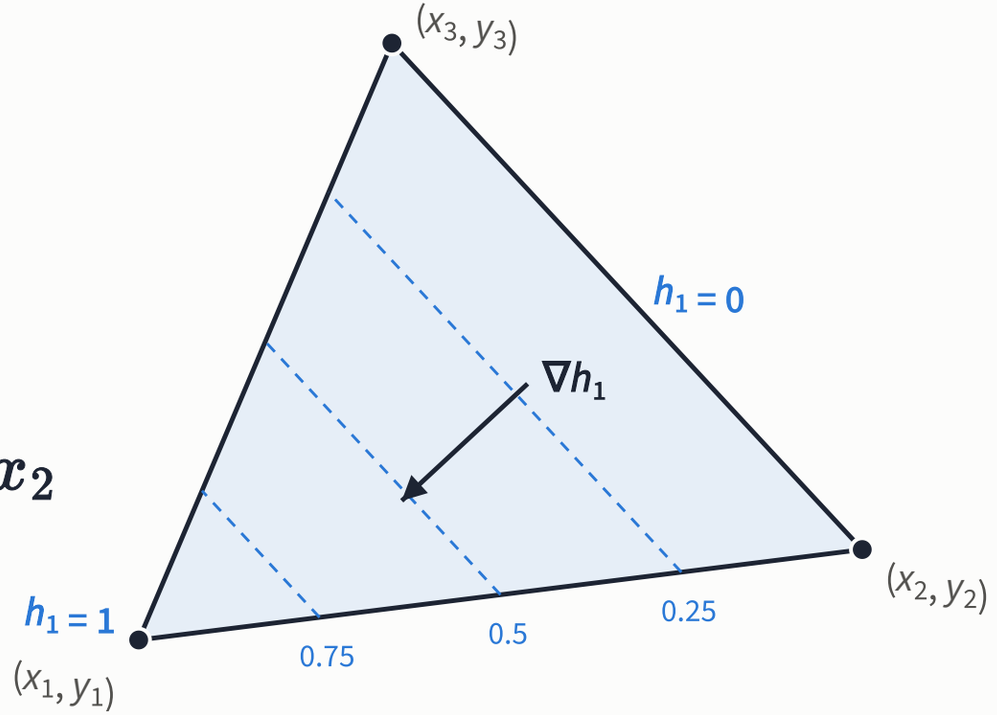
On a triangle with the vertices $(x_\alpha, y_\alpha), \alpha = 1, 2, 3$ (counterclockwise), u_h is linear and fixed by the three nodal values:

$$u_h = \sum_{\alpha=1}^{3} U_\alpha \, h_\alpha, \qquad h_\alpha(x, y) = \frac{a_\alpha + b_\alpha \, x + c_\alpha \, y}{2A}$$

$$a_1 = x_2 \, y_3 - x_3 \, y_2, \qquad b_1 = y_2 - y_3, \qquad c_1 = x_3 - x_2$$

and the other coefficients by cyclic permutation of the indices; A is the area of the triangle.

$h_\alpha(x_\beta, y_\beta) = \delta_{\alpha\beta}$ and $h_1 + h_2 + h_3 = 1$ (area coordinates): as in 1D, h_α is the hat function of the local node α, restricted to the triangle.



Element matrices of the CST

Constant gradients

$$\nabla h_\alpha = \frac{1}{2A}\begin{pmatrix} b_\alpha \\ c_\alpha \end{pmatrix}$$

constant on the triangle, hence the name **constant strain triangle** (exercise 2)

Element stiffness matrix

$$k^e_{\alpha\beta} = \int_{T_e} \nabla h_\alpha \cdot \nabla h_\beta \, dx = \frac{b_\alpha\, b_\beta + c_\alpha\, c_\beta}{4A}$$

Element load vector, constant f

$$f^e_\alpha = \int_{T_e} f\, h_\alpha \, dx = \frac{fA}{3}$$

The global basis function h_i of node i is a **pyramid** of height 1 over the **patch** of triangles around node i.

Assembly works as in 1D, with the coincidence table of the mesh: three global node numbers per element, and k^e is 3×3.

A uniformly heated square plate

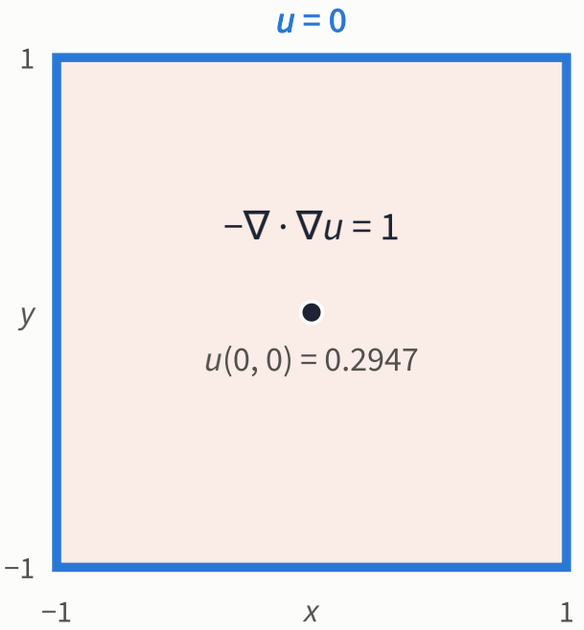
The edges are kept at temperature 0:

$$-\nabla \cdot \nabla u = 1 \quad \text{in } \Omega = (-1,1)^2, \qquad u = 0 \quad \text{on } \partial\Omega$$

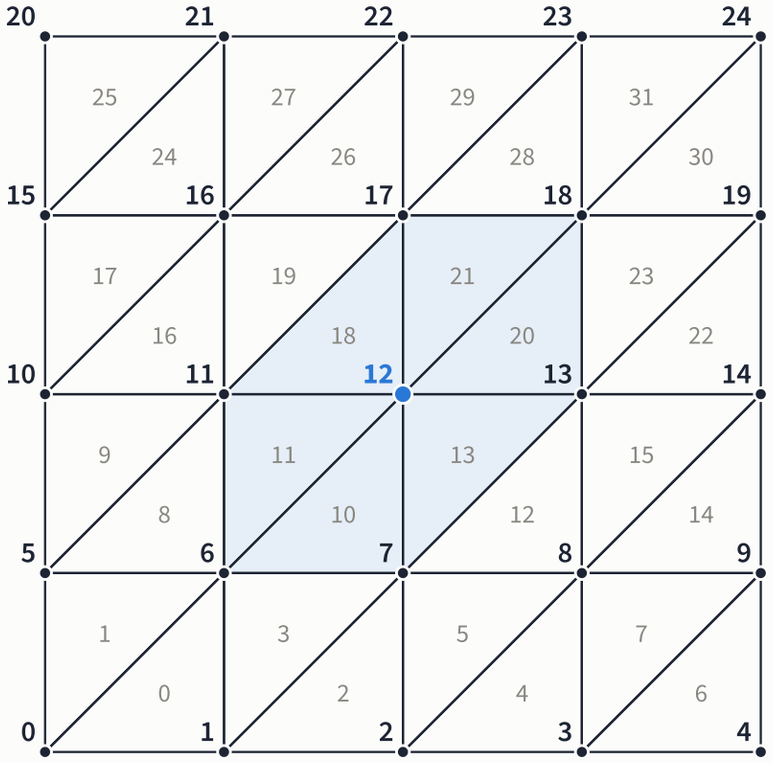
The temperature at the centre follows from a Fourier series (exercise 4):

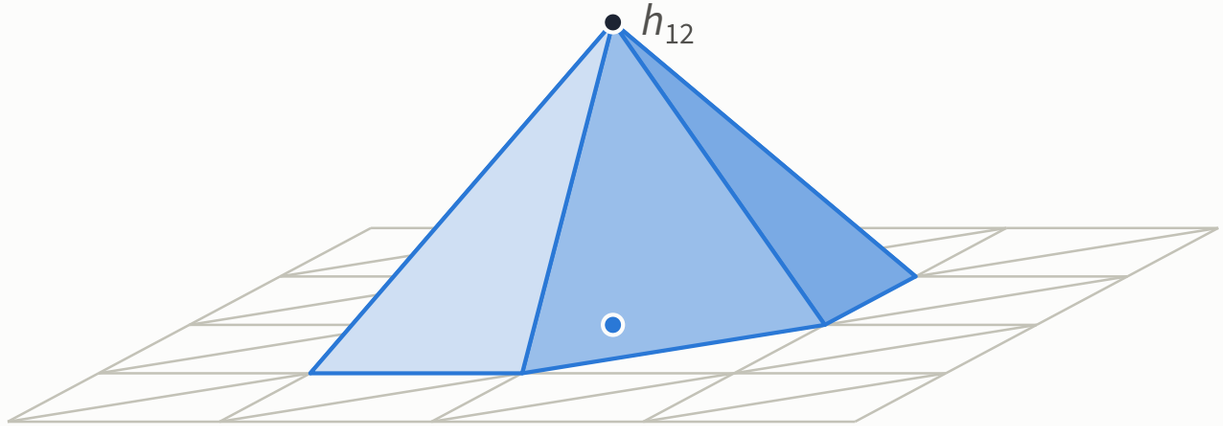
$$u(0,0) = \frac{1}{2} - \frac{16}{\pi^3} \sum_{k=1,3,5,\ldots} \frac{(-1)^{(k-1)/2}}{k^3 \cosh(k\pi/2)} = 0.294685413\ldots$$

This exact value measures the error of the FEM solutions in examples 3 and 5.



A global basis function on a triangulation






```
mesh = RectangleMesh(Point(-1.0, -1.0), Point(1.0, 1.0), 4, 4)
V    = FunctionSpace(mesh, 'Lagrange', 1)
tri  = mesh.cells()           # coincidence table
v2d  = vertex_to_dof_map(V)   # node -> its basis function

h_i = Function(V)
h_i.vector()[v2d[12]] = 1.0
```

Node numbers black, element numbers grey: `tri[10]` = [6, 7, 12]. The patch of node 12 consists of the elements 10, 11, 13, 18, 20 and 21.

Element matrix, assembled matrix and solution

```python
u, v = TrialFunction(V), TestFunction(V)
a = inner(grad(u), grad(v))*dx
L = Constant(1.0)*v*dx

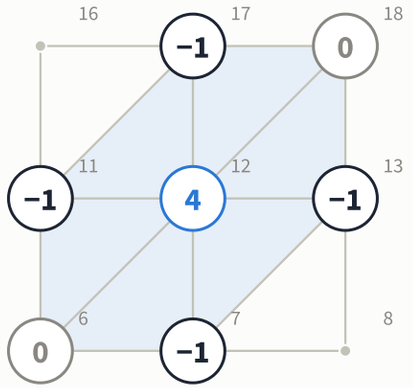
print(assemble_local(a, Cell(mesh, 0)))
K = assemble(a).array()
print(K[v2d[12], v2d])          # row of node 12

bc = DirichletBC(V, Constant(0.0), 'on_boundary')
uh = Function(V)
solve(a == L, uh, bc)
print(uh(0.0, 0.0))
```

Centre temperature $u_h(0,0) = 0.2813$, exact 0.2947: with only 9 unknowns, about 5 % too low.

Element 0, nodes 0, 1, 6:

$$k^e = \begin{pmatrix} 0.5 & -0.5 & 0 \\ -0.5 & 1 & -0.5 \\ 0 & -0.5 & 0.5 \end{pmatrix}$$



Row of node 12: 4 on the diagonal, -1 for the four neighbours: on right-angled triangles the CST gives the five-point finite difference stencil. The diagonal neighbours 6 and 18 get 0 (exercise 1.4).

Sparsity

$K_{ij} = a(h_i, h_j)$ is nonzero only if the nodes i and j belong to a common triangle. A row of K has about seven nonzero entries, independent of the size of the mesh. For n unknowns:

	stored entries	memory for $n = 10^4$	memory for $n = 10^6$
dense matrix	n^2	0.8 GB	8 TB
sparse matrix (CSR)	$\approx 7n$	0.8 MB	84 MB

Direct solvers: fill-in

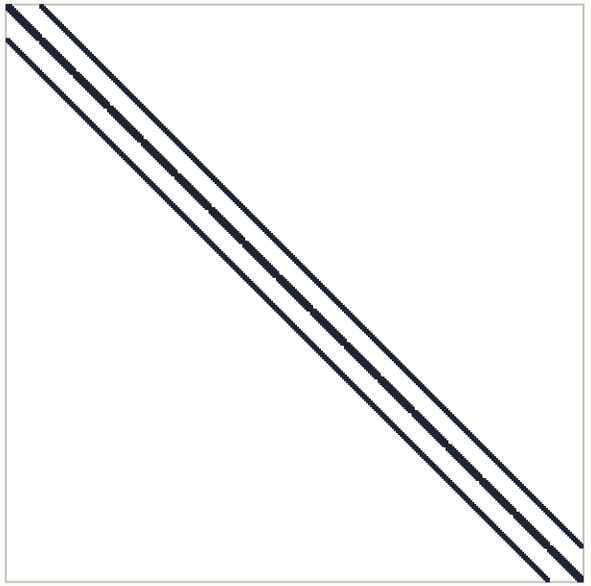
Gaussian elimination creates new nonzero entries in the triangular factors of K. Even with reordering, a sparse direct solver needs about $n^{1.5}$ operations in 2D and n^2 in 3D.

Iterative solvers

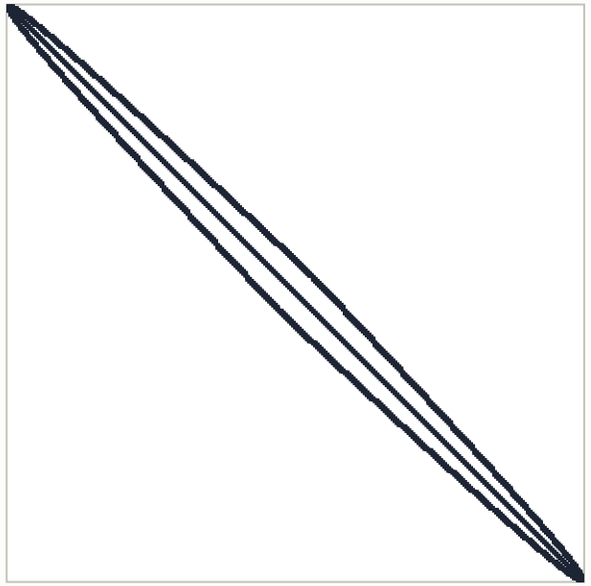
only need products $K\,p$: about $7n$ operations each, and K is never changed.

Sparsity pattern

node numbering



FEniCS numbering



n	nnz	nnz per row	n^2
289	1889	6.5	8.4e4
4225	29057	6.9	1.8e7
66049	460289	7.0	4.4e9
1050625	7346177	7.0	1.1e12

```
K = assemble(inner(grad(u), grad(v))*dx)
K.nnz()                    # stored entries
pyplot.spy(K.array())      # nonzero entries
```

The stiffness matrix of a 16×16 mesh, 289 nodes, in the order of the node numbers and in the order FEniCS uses. The numbering changes the pattern, not the number of entries.

nnz counts the stored entries, including the two zeros of the diagonal neighbours per row (example 3); the pattern shows only the nonzero ones.

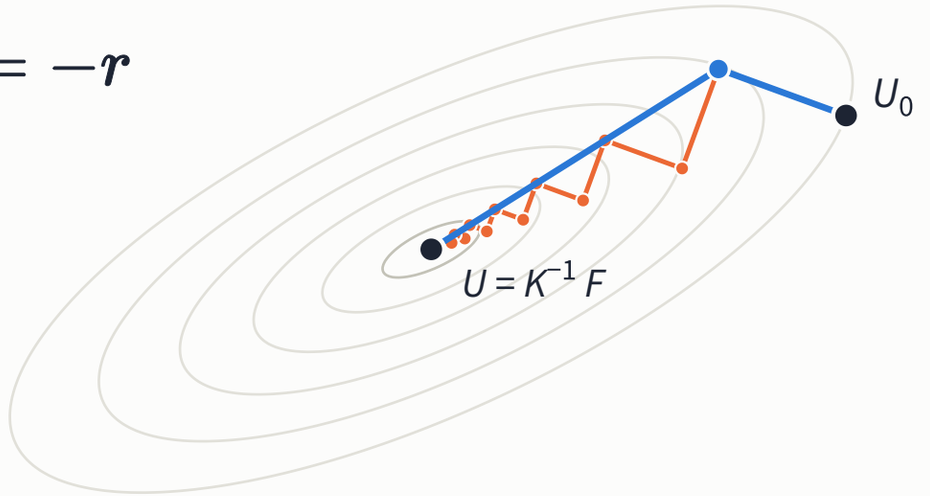
Solving $K\,U = F$ means minimizing

K is symmetric and positive definite. Solving $K\,U = F$ is then the same as minimizing

$$\Phi(U) = \tfrac{1}{2}\,U^T K\,U - U^T F, \qquad \nabla\Phi(U) = K\,U - F = -r$$

with the **residual** $r = F - K\,U$.

- **Steepest descent:** $U_{k+1} = U_k + \alpha_k\,r_k$ with $\alpha_k = r_k^T r_k / r_k^T K\,r_k$. It zigzags for an ill-conditioned matrix.
- **Conjugate gradient (CG) method:** the search directions p_k are K**-conjugate**, $p_j^T K\,p_k = 0$ for $j \neq k$. After k steps U_k minimizes Φ over $U_0 + \mathbf{span}\{r_0, Kr_0, \ldots, K^{k-1}r_0\}$ (Krylov space); the solution is reached after at most n steps.



The conjugate gradient method

$$
\begin{aligned}
&r_0 = F - K\,U_0, \quad p_0 = r_0 \\
&\text{for } k = 0, 1, 2, \ldots \\
&\qquad \alpha_k = r_k^T r_k \,/\, p_k^T K\,p_k \\
&\qquad U_{k+1} = U_k + \alpha_k\,p_k, \qquad r_{k+1} = r_k - \alpha_k\,K\,p_k \\
&\qquad \text{stop if } \|r_{k+1}\| \le \text{tol}\,\|F\| \\
&\qquad \beta_k = r_{k+1}^T r_{k+1} \,/\, r_k^T r_k, \qquad p_{k+1} = r_{k+1} + \beta_k\,p_k
\end{aligned}
$$

- Each iteration needs one product $K\,p_k$: about $7n$ operations.
- K is never changed, so there is no fill-in.
- The new direction p_{k+1} is the residual, made K-conjugate to p_k by adding $\beta_k\,p_k$.

Convergence and preconditioning

With the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ of K:

$$\|U - U_k\|_K \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|U - U_0\|_K, \qquad \|e\|_K = \sqrt{e^T K\, e}$$

For element size h, $\kappa \sim h^{-2}$: the number of iterations grows like $1/h$, in 2D like $\sqrt{n}$, and the total work like $n^{1.5}$.

Preconditioning

Solve $M^{-1}K\,U = M^{-1}F$ with $M \approx K$ cheap to invert: the diagonal of K (Jacobi), incomplete Cholesky.

Algebraic multigrid (AMG)

The work is proportional to n: the number of iterations stays constant. In FEniCS:

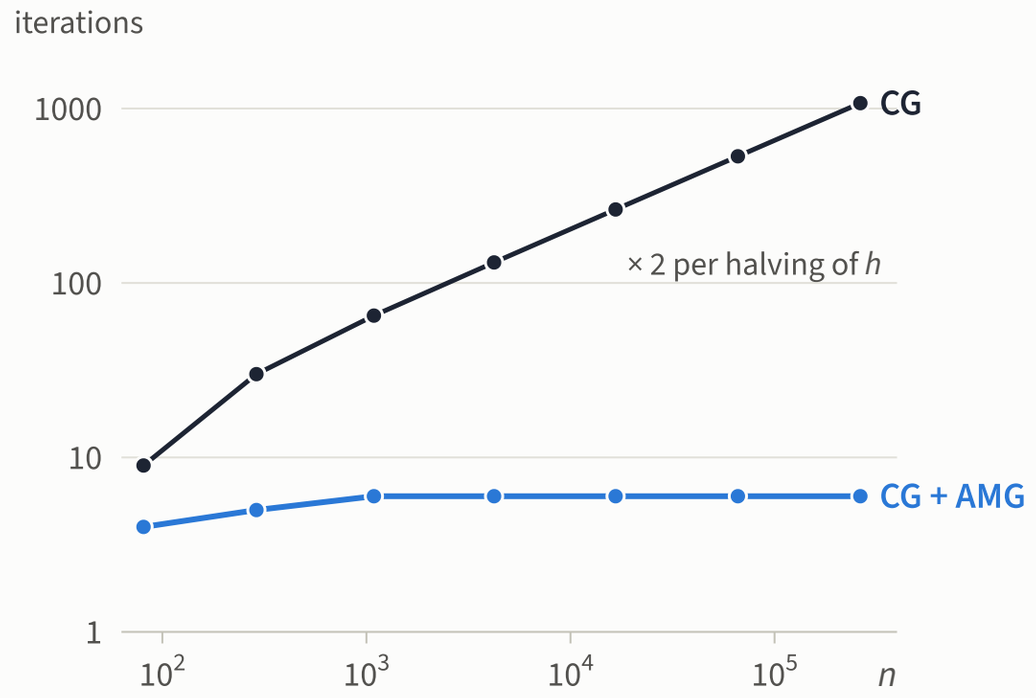
```
solver_parameters={'linear_solver': 'cg',
                   'preconditioner': 'amg'}
```

CG for the heated plate

```python
for N in [8, 16, 32, 64, 128, 256, 512]:
    mesh = RectangleMesh(Point(-1.0, -1.0), Point(1.0, 1.0), N, N)
    V    = FunctionSpace(mesh, 'Lagrange', 1)
    u, v = TrialFunction(V), TestFunction(V)
    bc   = DirichletBC(V, Constant(0.0), 'on_boundary')
    K, F = assemble_system(inner(grad(u), grad(v))*dx,
                           Constant(1.0)*v*dx, bc)

    for preconditioner in ['none', 'amg']:
        uh = Function(V)
        solver = KrylovSolver(K, 'cg', preconditioner)
        solver.parameters['relative_tolerance'] = 1e-10
        iterations = solver.solve(uh.vector(), F)
```

The residual is reduced by a factor 10^{10}. The error of the centre temperature drops from $3.6 \cdot 10^{-3}$ ($N = 8$) to $8.9 \cdot 10^{-7}$ ($N = 512$): like h^2.



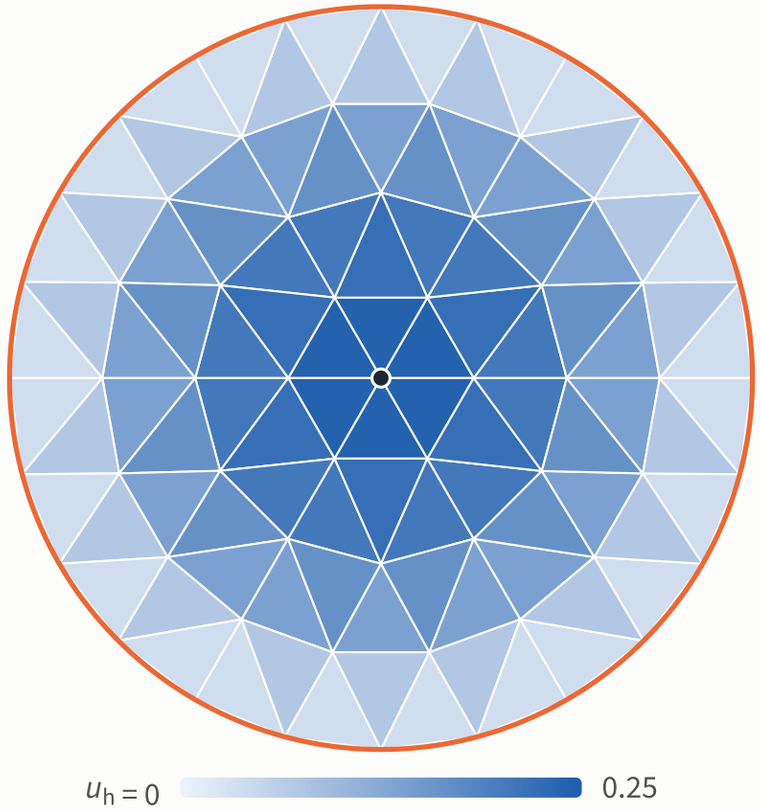
Without preconditioner the iterations double with every halving of h; with AMG they stay at about 6.

The mesh approximates the domain

Heated disk of radius 1, $-\nabla \cdot \nabla u = 1$, $u = 0$ on the boundary.
Exact: $u = (1 - r^2)/4$, centre temperature $1/4$.

```
mesh = UnitDiscMesh.create(MPI.comm_world, n, 1, 2)
V    = FunctionSpace(mesh, 'Lagrange', 1)
u, v = TrialFunction(V), TestFunction(V)
bc   = DirichletBC(V, Constant(0.0), 'on_boundary')
uh   = Function(V)
solve(inner(grad(u), grad(v))*dx == Constant(1.0)*v*dx,
      uh, bc, solver_parameters={'linear_solver': 'cg',
                                 'preconditioner': 'amg'})
```

h	0.620	0.337	0.175	0.089	0.045	0.023
n	19	61	217	817	3169	12481
error of $u_h(0,0)$	1.6e-3	7.3e-4	2.5e-4	8.0e-5	2.5e-5	6.7e-6



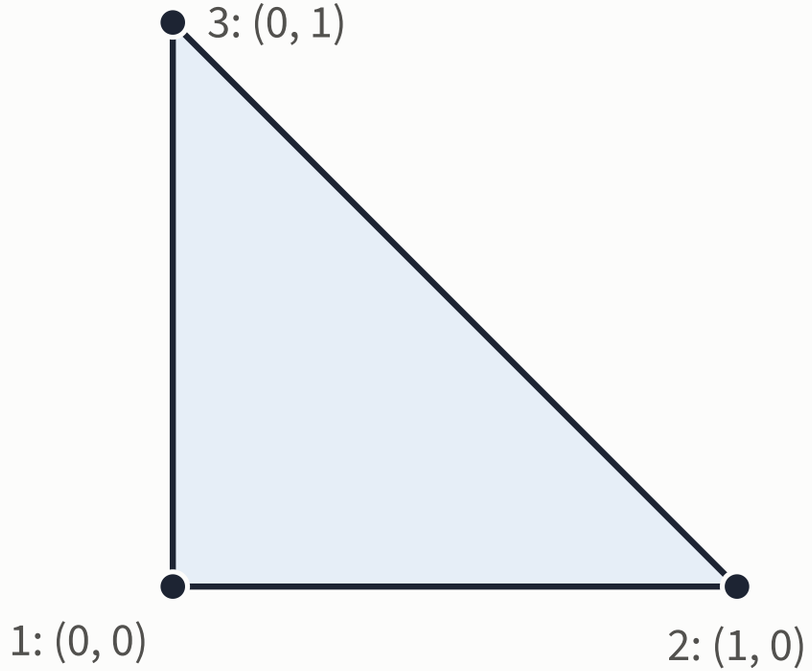
`UnitDiscMesh` replaces the boundary circle (orange) by a polygon; $n = 4$, 61 nodes. The error has two sources, the approximation of u and of Ω; both decrease like h^2.

The finite element method in four steps

1. Mesh the domain: nodes, elements T_e and the coincidence table (`mesh.cells()`).
2. One local basis function per node with $h_i(x_j) = \delta_{ij}$, so $U_i = u_h(x_i)$: hat functions in 1D, pyramids over the patch on triangles.
3. Compute the element matrices k^e, f^e and add them into K and F with the coincidence table. K is sparse, about 7 entries per row.
4. Solve $K\,U = F$ with CG. Without preconditioner the iterations grow like $1/h$; with AMG they stay constant: `{'linear_solver': 'cg', 'preconditioner': 'amg'}`.

The CST by hand

1. Compute h_1, h_2, h_3 on the triangle with the vertices $(0, 0)$, $(1, 0)$ and $(0, 1)$ and check $h_\alpha(x_\beta, y_\beta) = \delta_{\alpha\beta}$ and $h_1 + h_2 + h_3 = 1$.
2. Compute k^e and compare with `assemble_local`. Why is every row sum zero?
3. How does k^e change if the triangle is scaled by a factor s?
4. Show that $k^e_{\alpha\beta} = -\frac{1}{2}\cot\gamma$ for $\alpha \neq \beta$, where γ is the angle at the third vertex. Why are the entries of the diagonal neighbours in example 3 zero?



The CST in elasticity (theory)

In plane strain (lecture 3) the displacement $u = (u_x, u_y)$ is linear on the triangle, with the six nodal unknowns

$$U^e = (u_{x1}, u_{y1}, u_{x2}, u_{y2}, u_{x3}, u_{y3}).$$

1. Write $(\varepsilon_{xx}, \varepsilon_{yy}, 2\varepsilon_{xy}) = B\,U^e$ with a 3×6 matrix B built from b_α, c_α and A. Why is the strain constant?
2. With $\sigma = C\,\varepsilon$, show that $k^e = A\,B^T C\,B$.
3. In a bent cantilever the exact strain varies linearly over the height. How many CST elements over the height are needed? What does this suggest about coarse CST meshes in bending?

EXERCISE 3

Conjugate gradients and preconditioners

1. Perform two steps of steepest descent and two steps of CG for

$$K = \begin{pmatrix} 3 & 2 \\ 2 & 6 \end{pmatrix}, \qquad F = \begin{pmatrix} 2 \\ -8 \end{pmatrix}, \qquad U_0 = (-2, -2).$$

   Check that CG reaches the solution $(2, -2)$ and that the two search directions are K-conjugate. Sketch the level sets of Φ and both paths.

2. Repeat example 5 with the preconditioners `'jacobi'` and `'ilu'`. How do the iterations grow with N? Why does Jacobi not change anything here? (Look at the diagonal of K in example 3.)

The exact centre temperature

1. Show that $w = (1 - x^2)/2$ satisfies $-\nabla \cdot \nabla w = 1$ and $w = 0$ at $x = \pm 1$, and that $u - w$ is harmonic, zero at $x = \pm 1$ and equal to $-(1 - x^2)/2$ at $y = \pm 1$.
2. Expand $(1 - x^2)/2 = \sum_{k\ \text{odd}} c_k \cos(k\pi x/2)$ and show that $c_k = 16\,(-1)^{(k-1)/2}/(k\pi)^3$.
3. Solve for $u - w$ with the ansatz $\cos(k\pi x/2)\, g_k(y)$ and derive the formula for $u(0,0)$. Why does the series converge so fast?
4. From the errors in example 5, determine the exponent p in $\text{error} \approx C\, n^{-p}$. How many unknowns are needed for an error of 10^{-10}, and how much memory does the sparse matrix take?